

# Programozás Alapjai

Dr. Gergely Tamás  
Dr. Kicsi András

Szegedi Tudományegyetem  
Informatikai Intézet  
Szoftverfejlesztés Tanszék

2025

(v1207)



# Tartalomjegyzék

- 
- 1 Bemutatók**
    - Kurzus információk
    - Infrastruktúra szabályzatok
    - Mentorálás
  - 2 Python alapok**
    - Bevezető
    - Pénzváltás (1. verzió)
    - Pénzváltás (2. verzió)
  - 3 Alapfogalmak**
    - Alapfogalmak
    - Alapvető programelemek fogalmak
  - 4 Python feltételes ciklusok**
    - Röppálya számítás
    - Röppálya szimuláció
  - 5 Szoftverfejlesztési életciklus modellek**
    - A programozás fázisai
  - 6 Python sztringek**
    - Python sztringek
    - Basename meghatározása
  - 7 Vezérlési szerkezetek**
    - Bevezetés
    - Szekvenciális vezérlés
    - Függvények
  - 8 Python listák**
    - Python list és tuple adattípusok
    - Python listáshoz adattípusok műveletei
    - Basename és eltelt idő
  - 9 Vezérlési szerkezetek**
    - Szelekciós vezérlések
  - 10 Python iterációk**
    - Csúszóátlag számítása
  - 11 Vezérlési szerkezetek**
    - Ismétléses vezérlések
  - 12 Python halmazok, comprehension**
    - Python set és frozenset adattípusok
    - Tér legtovábbi pontjai
  - 13 Vezérlési szerkezetek**
    - Ismétléses vezérlések
  - 14 Python dictionary, függvény mint típus**
    - Python dict adattípus
    - Python comprehension
    - Python function adattípus
  - 15 Vezérlési szerkezetek**
    - Eljárásvezérlés
  - 16 Python fájlkezelés**
    - Fájlkezelés
  - 17 Vezérlési szerkezetek**
    - Eljárásvezérlés
  - 18 Python formátált IO**
    - Python formátált IO
  - 19 Adatkezelés, adattárolás**
    - Az adatkezelés szintjei
    - Az adattárolás jellemzői
  - 20 Python típusok**
    - Python numerikus típusok
  - 21 Absztrakt adattípusok**
    - Egyszerű adattípusok
    - Összetett adattípusok
  - 22 Python típusok**
    - További Python típusok
    - Python function adattípus
  - 23 Absztrakt adattípusok**
    - Összetett adattípusok
  - 24 Tesztelés**
    - Tesztelés
    - Tesztelés Python-ban
    - A pytest
  - 25 Vizualizáció és egyéb modulok**
    - Fájlkezelő modulok
    - Vizualizáció
  - 26 More Python**
    - A Python program végrehajtásának folyamata
    - Függvényparaméterek és argumentumok
    - Python unpacking
    - Kivételkezelés

# Tartalomjegyzék

- 
- 1 Bemutatók**
    - Kurzus információk
    - Infrastruktúra szabályzatok
    - Mentorálás
  - 2 Python alapok**
    - Bevezető
    - Példák (1. verzió)
    - Példák (2. verzió)
  - 3 Alapfogalmak**
    - Alapfogalmak
    - Alapvető programozási fogalmak
  - 4 Python feltételes ciklusok**
    - Röppálya számítás
    - Röppálya szimuláció
  - 5 Szoftverfejlesztési életciklus modellek**
    - A programozás fázisai
  - 6 Python sztringek**
    - Python sztringek
    - Basename meghatározása
  - 7 Vezérlési szerkezetek**
    - Bevezetés
    - Szekvenciális vezérlés
    - Függvények
  - 8 Python listák**
    - Python list és tuple adattípusok
    - Python listák adattípusok műveletei
    - Basename és eltelt idő
  - 9 Vezérlési szerkezetek**
    - Szelekciós vezérlések
  - 10 Python iterációk**
    - Csúszólag számítása
  - 11 Vezérlési szerkezetek**
    - Ismétléses vezérlések
  - 12 Python halmazok, comprehension**
    - Python set és frozenset adattípusok
    - Tér legtovábbi pontjai
  - 13 Vezérlési szerkezetek**
    - Ismétléses vezérlések
  - 14 Python dictionary, függvény mint típus**
    - Python dict adattípus
    - Python comprehension
    - Python function adattípus
  - 15 Vezérlési szerkezetek**
    - Eljárásvezérlés
  - 16 Python fájlkezelés**
    - Fájlkezelés
  - 17 Vezérlési szerkezetek**
    - Eljárásvezérlés
  - 18 Python formátált IO**
    - Python formátált IO
  - 19 Adatkezelés, adattárolás**
    - Az adatkezelés szintjei
    - Az adattárolás jellemzői
  - 20 Python típusok**
    - Python numerikus típusok
  - 21 Absztrakt adattípusok**
    - Egyszerű adattípusok
    - Összetett adattípusok
  - 22 Python típusok**
    - További Python típusok
    - Python function adattípus
  - 23 Absztrakt adattípusok**
    - Összetett adattípusok
  - 24 Tesztelés**
    - Tesztelés
    - Tesztelés Python-ban
    - A pytest
  - 25 Vizualizáció és egyéb modulok**
    - Fájlkezelő modulok
    - Vizualizáció
  - 26 More Python**
    - A Python program végrehajtásának folyamata
    - Függvényparaméterek és argumentumok
    - Python unpacking
    - Kivételkezelés

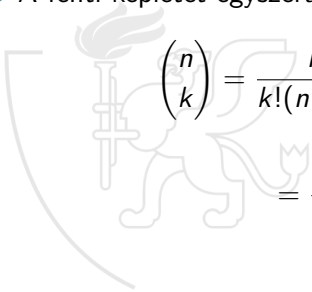
# $\binom{n}{k}$ kiszámítása

## Problémafelvetés és specifikáció

- Problémafelvetés
  - Számítsuk ki  $\binom{n}{k}$  értékét!
- Specifikáció
  - A probléma bemenete két egész szám,  $n$  és  $k$ .
  - A probléma kimenete  $\binom{n}{k} = \frac{n!}{k!(n-k)!}$  ha értelmezhető  $n$ -re és  $k$ -ra, különben 0.



- Készítsünk az  $\binom{n}{k}$  kiszámolására egy függvényt.
- Ismeretes, hogy  $\binom{n}{k} = \frac{n!}{k!(n-k)!}$ , de az is nyilvánvaló, hogy nem célszerű a számolást a faktoriálisok számításával kezdeni majd az osztás és szorzás műveletet ezekre elvégezni, mert egyszerűsítéssel a számlálóból és nevezőből is rengeteg tag kieshet.
- A fenti képletet egyszerűsítve és átalakítva kapjuk, hogy


$$\begin{aligned}\binom{n}{k} &= \frac{n!}{k!(n-k)!} = \frac{n \cdot (n-1) \cdot \dots \cdot (n-k+1) \cdot (n-k) \cdot \dots \cdot 1}{k \cdot (k-1) \cdot \dots \cdot 1 \cdot (n-k) \cdot \dots \cdot 1} \\ &= \frac{n \cdot (n-1) \cdot \dots \cdot (n-k+1)}{1 \cdot 2 \cdot \dots \cdot k} = \prod_{i=1}^k \frac{n-i+1}{i}\end{aligned}$$

- Számlálós ismétléses vezérlésről akkor beszélünk, ha a ciklusmagot végre kell hajtani egy változó sorban minden olyan értékére (növekvő vagy csökkenő sorrendben), amely egy adott intervallumba esik.
- Legyen  $a$  és  $b$  egész érték,  $i$  egész típusú változó,  $M$  pedig tetszőleges művelet, amelynek nincs hatása  $a$ ,  $b$  és  $i$  értékére.



# Növekvő számlálós ismétléses vezérlés

- Az  $a$  és  $b$  határértékekből,  $i$  ciklusváltozóból és  $M$  műveletből (ciklusmagból) képzett növekvő számlálós ismétléses vezérlés az alábbi vezérlési előírást jelenti:
  - 1 Legyen  $i = a$ .
  - 2 Ha  $b < i$ , akkor az ismétlés és ezzel együtt az összetett művelet végrehajtása befejeződött.
  - 3 Egyébként, vagyis ha  $i \leq b$ , akkor hajtsuk végre az  $M$  műveletet.
  - 4 Növeljük  $i$  értékét 1-gyel, és folytassuk a 2.) lépéssel.



# Csökkenő számlálós ismétléses vezérlés

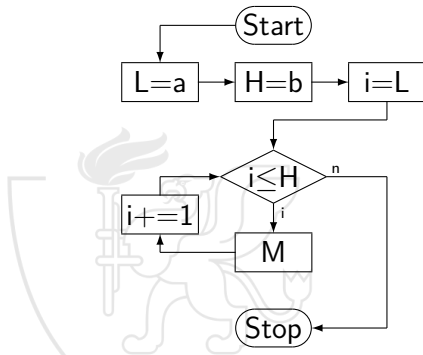
- Az  $a$  és  $b$  határértékekből,  $i$  ciklusváltozóból és  $M$  műveletből (ciklusmagból) képzett csökkenő számlálós ismétléses vezérlés az alábbi vezérlési előírást jelenti:
  - 1 Legyen  $i = b$ .
  - 2 Ha  $i < a$ , akkor az ismétlés és ezzel együtt az összetett művelet végrehajtása befejeződött.
  - 3 Egyébként, vagyis ha  $a \leq i$ , akkor hajtsuk végre az  $M$  műveletet.
  - 4 Csökkentsük  $i$  értékét 1-gyel, és folytassuk a 2.) lépéssel.



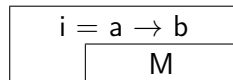
# Növekvő számlálásos ismétléses vezérlés

## Folyamatábra és struktogram

- Folyamatábrája:



- Struktogramja:

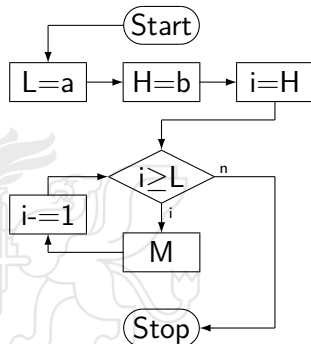


- A határértékek alapján előfordulhat, hogy az  $M$  műveletet egyszer sem hajtjuk végre.

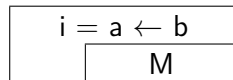
# Csökkenő számlálós ismétléses vezérlés

## Folyamatábra és struktogram

- Folyamatábrája:



- Struktogramja:

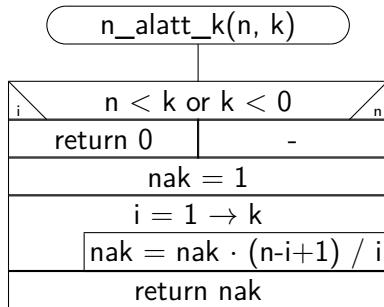


- A határértékek alapján előfordulhat, hogy az  $M$  műveletet egyszer sem hajtjuk végre.

# $\binom{n}{k}$ kiszámítása

## Algoritmustervezés – Struktogram

- Az  $\binom{n}{k}$  értékét számító függvény struktogramja:



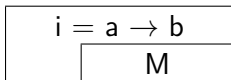
# A for utasítás

- Ha valamilyen műveletet több előre megadott értékére is végre kell hajtani, akkor a **for** utasítás használható.
- **for utasítás:** A **for** utasítás Python-ban egy **for** **blokk**-ból és utána nulla vagy egy **else** **blokk**-ból állhat  
(`[[[:for blokk:]] [[[:else blokk:]]?]`).
- **for blokk:** A `'for'` kulcsszó, utána egy változó, az `'in'` kulcsszó, egy (iterálható) kifejezés és egy `':'`, majd a következő sortól egy indentált kódblokk  
(`'for' [[[:változó:]] 'in' [[[:Kifejezés:]] ':' '\n' [[[:Indentált kódblokk:]]]`).
- **else blokk:** Az `'else'` kulcsszó és egy `':'` majd a következő sortól egy indentált kódblokk  
(`'else' ':' '\n' [[[:Indentált kódblokk:]]]`).

# Növekvő számlálásos ismétléses vezérlés

Megvalósítás Python nyelven

- Struktogramja:



- Megvalósítása:

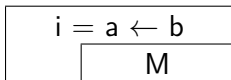
```
for i in range(a, b + 1):  
    M
```



# Csökkenő számlálós ismétléses vezérlés

Megvalósítás Python nyelven

- Struktogramja:



- Megvalósítása:

```
for i in range(b, a - 1, -1):  
    M
```



# $\binom{n}{k}$ ciklusos implementáció [1/2]

nalattk\_f.py [1-16]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTIK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2024-05-31
6cc-by-nc-nd
7
8n_alatt_k_kiszámítása nemrekurzív módon
9"""
10def n_alatt_k(n, k):
11    if n < k or k < 0:
12        return 0
13    nak = 1
14    for i in range(1, k + 1):
15        nak = nak * (n - i + 1) // i
16    return nak
```

# $\binom{n}{k}$ ciklusos implementáció [2/2]

nalattk\_f.py [18–29]

```
18 def main():
19     print("Kérem_n-t_és_k-t_az_n_alatt_k_értékének_kiszámításhoz:")
20     n = int(input("n:_"))
21     k = int(input("k:_"))
22     nak = n_alatt_k(n, k)
23     if nak:
24         print(f"{n}_alatt_{k}_={nak}")
25     else:
26         print(f"{n}_alatt_{k}_nem_értelmezett!")
27
28 if __name__ == "__main__":
29     main()
```

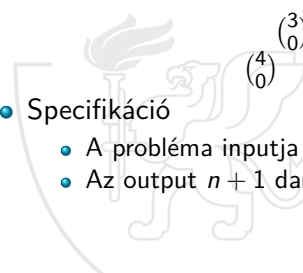


# Pascal háromszög

## Problémafelvetés és specifikáció

- Problémafelvetés

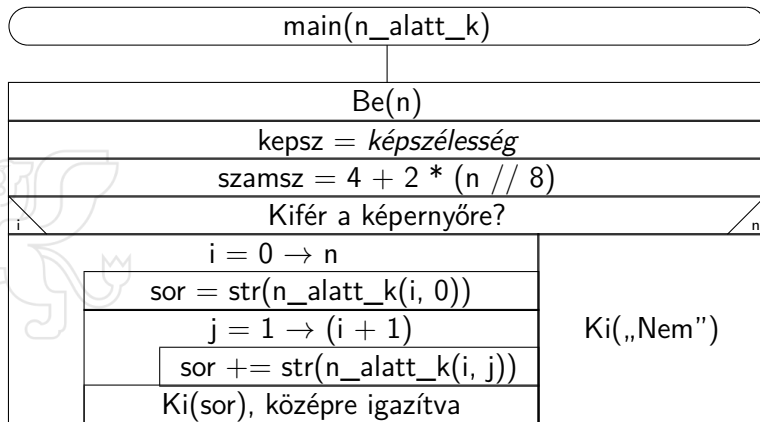
- A Pascal háromszög kiszámolása és megjelenítése.


$$\begin{array}{ccccc} & & \binom{0}{0} & & \\ & & \binom{1}{0} & & \binom{1}{1} \\ & & \binom{2}{0} & & \binom{2}{1} & & \binom{2}{2} \\ & & \binom{3}{0} & & \binom{3}{1} & & \binom{3}{2} & & \binom{3}{3} \\ & & \binom{4}{0} & & \binom{4}{1} & & \binom{4}{2} & & \binom{4}{3} & & \binom{4}{4} \end{array} = \begin{array}{cccccc} & & & & 1 & & & & \\ & & & & 1 & & 1 & & \\ & & & 1 & & 2 & & 1 & \\ & & 1 & & 3 & & 3 & & 1 \\ 1 & & 4 & & 6 & & 4 & & 1 \end{array}$$

- Specifikáció

- A probléma inputja egy  $n$  nemnegatív egész szám.
  - Az output  $n + 1$  darab sor piramisszerűen rendezve, rendre a Pascal háromszög sorai.

- A főprogram feladata a beolvasás, input ellenőrzése, majd a `n_alatt_k` függvényt alkalmas paraméterekkel aktivizálva a soronkénti kiíratás.



# Pascal háromszög ciklusos $\binom{n}{k}$ implementációval

P3szog\_f.py [1–14]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2024-05-31
6cc-by-nc-nd
7
8Pascal háromszög kirajzolása nemrekurzív n alatt k függvényel
9"""
10from nalattk_f import n_alatt_k
11from P3szog_m import main
12
13if __name__ == "__main__":
14    main(n_alatt_k)
```

# Pascal háromszög ciklusos $\binom{n}{k}$ implementációval

P3szog\_m.py [1–25]

```
1#!/usr/bin/env python3
2"""
3Programozás_Alapjai_eloadas_peldaprogram
4SZTE_TTIK,_Informatikai_Intezet,_Szoftverfejlesztés_Tanszék
5Gergely_Tamás,_gertom@inf.u-szeged.hu,_2024-05-31
6cc-by-nc-nd
7
8Pascal_háromszög_kirajzolása,_main_függvény
9"""
10from os import environ
11
12def main(n_alatt_k):
13    n = int(input("Kérem a Pascal háromszög sorainak számát: "))
14
15    kepsz = int(environ['COLUMNS']) if 'COLUMNS' in environ else 80
16    szamsz = 4 + 2 * (n // 8)
17
18    if 0 <= n <= ((kepsz - 2) // szamsz):
19        for i in range(0, n + 1):
20            sor = "%*d" % (szamsz, n_alatt_k(i, 0))
21            for j in range(1, i + 1):
22                sor += "%*d" % (szamsz, n_alatt_k(i, j))
23            print(sor.center(kepsz - 2))
24    else:
25        print(f"{n} sor nem fér ki a képernyőre!")
```

# Pascal háromszög ciklusos $\binom{n}{k}$ implementációval

nalattk\_f.py [1–16]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTIK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2024-05-31
6cc-by-nc-nd
7
8n_alatt_k_kiszámítása nemrekurzív módon
9"""
10def n_alatt_k(n, k):
11    if n < k or k < 0:
12        return 0
13    nak = 1
14    for i in range(1, k + 1):
15        nak = nak * (n - i + 1) // i
16    return nak
```

# Számsorozat legnagyobb közös osztója

## Problémafelvetés és specifikáció

- Problémafelvetés
  - Számítsuk ki pozitív egész számok sorozatának legnagyobb közös osztóját!
- Specifikáció
  - A probléma inputja egy pozitív egész számokból álló sorozat, melyet a 0 zár.
  - A számsorozat elemeinek legnagyobb közös osztója.



# Számsorozat legnagyobb közös osztója

## Algoritmustervezés

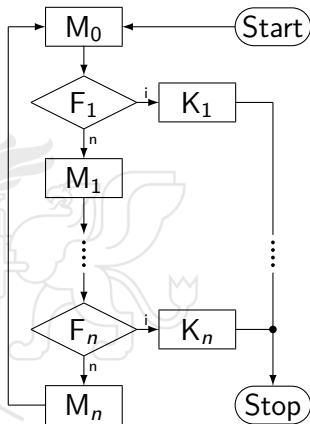
- Az algoritmus lényege egy olyan ismétléses vezérlés, amely ciklusmagjának egyszeri végrehajtása kiszámítja a már feldolgozott input számsor és a beolvasott következő szám legnagyobb közös osztóját.
  - Ez azt jelenti, hogy minden cikluslépésben ki kell számolnunk két szám legnagyobb közös osztóját.
- Látható, hogy a ciklusmag ismétlését célszerű a ciklusmagban vezérelni, mert az ismétlés befejeződhet úgy, hogy
  - végére értünk az inputnak, vagy
  - a feldolgozott sorozat legnagyobb közös osztója 1.

- Amikor a ciklusmag ismétlését a ciklusmagon belül vezéreljük úgy, hogy a ciklus különböző pontjain adott feltételek teljesülése esetén a ciklus végrehajtását befejezzük, hurok ismétléses vezérlésről beszélünk.
- Legyenek  $F_i$  logikai kifejezések,  $K_i$  és  $M_j$  pedig tetszőleges (akár üres) műveletek  $1 \leq i \leq n$  és  $0 \leq j \leq n$  értékekre. Az  $F_i$  kijárat feltételekből,  $K_i$  kijárat műveletekből és az  $M_j$  műveletekből képzett hurok ismétléses vezérlés a következő előírást jelenti:
  - 1 Az ismétléses vezérlés következő végrehajtandó egysége legyen az  $M_0$  művelet.
  - 2 Ha a végrehajtandó egység az  $M_j$  művelet, akkor ez végrehajtódik.
  - 3  $j = n$  esetén folytassuk az 1.) lépéssel.
  - 4 Különben legyen a következő végrehajtandó egység az  $F_{j+1}$  feltétel.
  - 5 Ha a végrehajtandó egység az  $F_i$  feltétel, akkor értékeljük ki.
  - 6 Ha  $F_i$  igaz volt, akkor hajtsuk végre a  $K_i$  műveletet, és fejezzük be a vezérlést.
  - 7 Különben legyen a következő végrehajtandó egység az  $M_i$  művelet, és folytassuk a 2.) lépéssel.

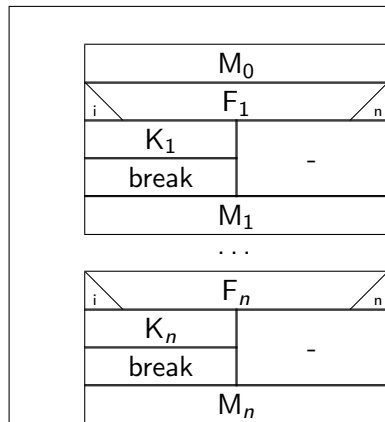
# Hurok ismétléses vezérlés

## Folyamatátbra és struktogram

- Folyamatátbraja:



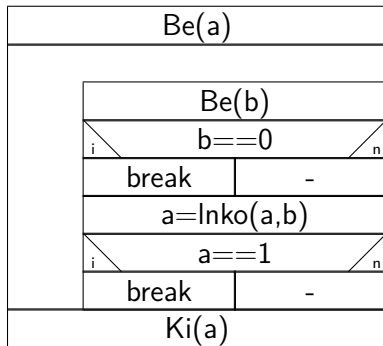
- Struktogramja:



# Számsorozat legnagyobb közös osztója

Algoritmustervezés – Struktogram

- A főprogram:



# Két szám legnagyobb közös osztója

## Problémafelvetés és specifikáció

- Problémafelvetés
  - Számítsuk ki két nemnegatív egész szám legnagyobb közös osztóját!
- Specifikáció
  - Készítsünk függvényt.
  - Az input  $x$  és  $y$  nemnegatív egész számok.
  - Az output egy egész szám,  $x$  és  $y$  legnagyobb közös osztója.



# Két szám legnagyobb közös osztója

## Algoritmustervezés

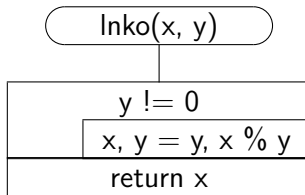
- Euklidesz algoritmusát valósítjuk meg, melynek alapja, hogy
  - ha  $a \geq b > 0$  akkor  $\text{luko}(a, b) = \text{luko}(a - b, b)$ , és
  - $\text{luko}(a, 0) = a$ , vagyis a nagyobbik számból a kisebbiket kivonogatva előbb-utóbb megkapjuk a két eredeti szám legnagyobb közös osztóját.
- A gyakorlatban az ismételt kivonogatás helyett maradékos osztást alkalmazunk, ami egyetlen lépésben megadja annak az  $a' - b$  kivonásnak az eredményét, amikor  $a' - b < b$  teljesül, vagyis  $b$  veszi át a nagyobb szám szerepét.



# Két szám legnagyobb közös osztója

## Algoritmustervezés – Struktogram

- A legnagyobb közös osztó függvény:



# A break és continue utasítások

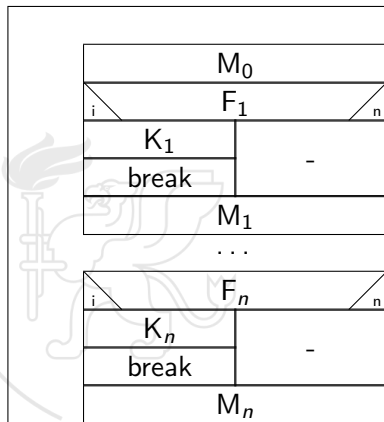
- A Python nyelvben a ciklusmag folyamatos végrehajtásának megszakítására két utasítás használható:
  - break** Megszakítja a ciklust, a program végrehajtása a ciklus utáni első utasítással folytatódik. A ciklushoz tartozó else ág **nem** hajtódik végre.
  - continue** Megszakítja a ciklusmag aktuális lefutását, a vezérlés a ciklus feltételének kiértékelésével (`while`) illetve a soron következő elem feldolgozásával (`for`) folytatódik.



# Hurok vezérlés

Megvalósítás Python nyelven

- A vezérlés struktogramja:



- Megvalósítása:

```
while True:
    M0
    if F1:
        K1
        break
    ...
    if Fn:
        Kn
        break
    Mn
```

# Számsorozat legnagyobb közös osztója [1/2]

Inko.py [1–13]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2024-05-31
6cc-by-nc-nd
7
8Számsorozat legnagyobb közös osztójának meghatározása
9"""
10def loko(x, y):
11    while y != 0:
12        x, y = y, x % y
13    return x
```

# Számsorozat legnagyobb közös osztója [2/2]

lnko.py [15–29]

```
15 def main():
16     print("A program pozitív egész számok legnagyobb közös osztóját számítja.")
17     print("Kérem a számok sorozatát, amit 0-zár!")
18     a = int(input("? "))
19     while True:
20         b = int(input("? "))
21         if b == 0:
22             break
23         a = lnko(a, b)
24         if a == 1:
25             break
26     print(f"A számok legnagyobb közös osztója: {a}")
27
28 if __name__ == "__main__":
29     main()
```

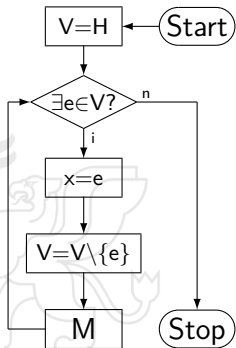
# Diszkrét ismétléses vezérlés

- Diszkrét ismétléses vezérlésről akkor beszélünk, ha a ciklusmagot végre kell hajtani egy halmaz minden elemére tetszőleges sorrendben.
- Legyen  $x$  egy  $T$  típusú változó,  $H$  a  $T$  értékkészletének részhalmaza,  $M$  pedig tetszőleges művelet, amelynek nincs hatása  $x$  és  $H$  értékére. A  $H$  halmazból,  $x$  ciklusváltozóból és  $M$  műveletből (ciklusmagból) képzett diszkrét ismétléses vezérlés az alábbi vezérlési előírást jelenti:
  - 1 Ha a  $H$  halmaz minden elemére végrehajtottuk az  $M$  műveletet, akkor vége a vezérlésnek.
  - 2 Egyébként vegyük a  $H$  halmaz egy olyan tetszőleges  $e$  elemét, amelyre még nem hajtottuk végre az  $M$  műveletet.
  - 3 Legyen  $x = e$  és hajtsuk végre az  $M$  műveletet, majd folytassuk az 1.) lépéssel.

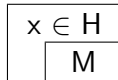
# Diszkrét ismétléses vezérlés

## Folyamatábra és struktogram

- Folyamatábrája:



- Struktogramja:

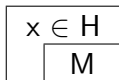


- A  $H$  halmaz számossága határozza meg, hogy az  $M$  művelet hányszor hajtódik végre.
- Ha a  $H$  az üres halmaz, akkor a diszkrét ismétléses vezérlés az  $M$  művelet végrehajtása nélkül befejeződik.

# Diszkrét ismétléses vezérlés

Megvalósítás Python nyelven

- Struktogramja:



- Megvalósítása

```
for x in H:  
    M
```



# Diszkrét ismétléses vezérlés

## Példák

- A diszkrét ismétléses vezérlés Python-ban nagyon kötődik az iterálható típusokhoz (`set`, `list`, `tuple`, `dict`, `range`), ezért példát majd ezen típusok ismertetésekor látunk rá

